Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Practical

Practical – VII

Roll No. : T045	Name: Omkar Thukarul
Class : TYIT	Date/Time of Submission : 5/08/2026

Aim: Machine Learning

2). Using a suitable regression dataset, implement Linear Regression using scikit-learn.

CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Load dataset
data = pd.read_csv("Student_Performance.csv")
# Input and output
X = data[['Hours']]
y = data['Marks']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Train model
model = LinearRegression()
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:",
      round(model.score(X_train, y_train)*100, 2))
print("Testing Accuracy:",
      round(model.score(X_test, y_test)*100, 2))
# Predict marks for 11 study hours
new_data = pd.DataFrame([[11]], columns=['Hours'])
pred = model.predict(new_data)
print("Predicted Marks for 11 hours:", round(pred[0], 2))
```

OUTPUT:

```
Training Accuracy: 99.64
Testing Accuracy: 98.71
Predicted Marks for 11 hours: 93.57
```

Extra). California Housing Dataset

Problem Statement

A real estate company wants to predict the median house value based on housing features.

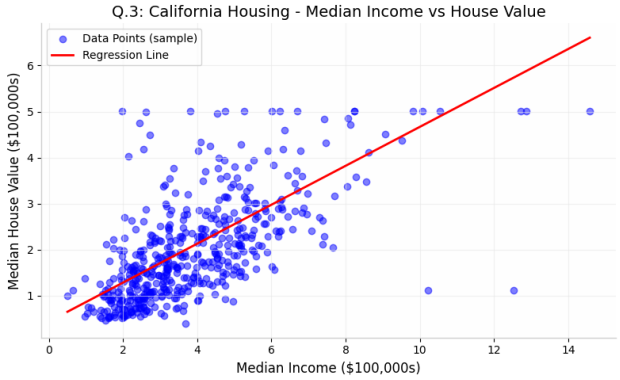
CODE: ***OUTPUT:***

```
import pandas as pd

from sklearn.datasets import fetch_california_housing
```

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt
# 1. Load dataset
housing = fetch_california_housing(as_frame=True)
df = housing.frame
df['MedHouseValue'] = housing.target
# 2. First five records
print(df.head())
# 3. Check missing values
print(df.isnull().sum())
# 4. Select Median Income as independent variable
X = df[["MedInc"]]
y = df["MedHouseValue"]
# 5. Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
# 6. Train model
model = LinearRegression()
model.fit(X_train, y_train)
# 7. Predict
y_pred = model.predict(X_test)
# 8. Metrics
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
print("R² Score:", r2_score(y_test, y_pred))
# 9. Plot
sample = df.sample(n=500, random_state=1)
plt.scatter(sample["MedInc"], sample["MedHouseValue"], alpha=0.5)
plt.plot(sample["MedInc"].sort_values(),
        model.predict(sample[["MedInc"]].sort_values()), color='red')
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
```

```
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  Longitude  MedHouseVal  MedHouseValue
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88    -122.23        4.526          4.526
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86    -122.22        3.585          3.585
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85    -122.24        3.521          3.521
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85    -122.25        3.413          3.413
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85    -122.25        3.422          3.422
MedInc          0
HouseAge        0
AveRooms        0
AveBedrms       0
Population      0
AveOccup        0
Latitude        0
Longitude       0
MedHouseVal     0
MedHouseValue   0
dtype: int64
Coefficient: 0.4226546718742174
Intercept: 0.4398905377109563
R² Score: 0.4676213698102699
Backend tkagg is interactive backend. Turning interactive mode on.
```



```
plt.show()

# 10. Interpretation

print("Positive correlation: Higher income areas have higher house values.")
```

2). Using the MNIST dataset (digit vs not digit), implement a binary classifier.

CODE:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
# Load MNIST-like digits dataset
digits = load_digits()
X = digits.data
y = digits.target
# Binary classification
# True if digit is 5, otherwise False
y = (y == 5)
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
# Accuracy
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
# Predict first test sample
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 5")
else:
    print("Digit is not 5")
```

OUTPUT:

```
Training Accuracy: 100.0
Testing Accuracy: 98.88888888888889
Digit is not 5
```

EXTRA)Q.1 Modify the program to detect Digit 7 vs Not Digit 7 instead of Digit 5 vs Not Digit 5.

CODE:

```
from sklearn.datasets import load_digits

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

digits = load_digits()

X = digits.data

y = digits.target

# Binary classification: Digit 7 vs Not Digit 7
```

OUTPUT:

```
Training Accuracy: 100.0
Testing Accuracy: 99.07407407407408
Digit is not 7
```

```
y = (y == 7)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 7")
else:
    print("Digit is not 7")
```

Q.2 Modify the program to detect Digit 0 vs Not Digit 0.

CODE:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
digits = load_digits()
X = digits.data
y = digits.target
# Binary classification: Digit 0 vs Not Digit 0
y = (y == 0)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 0")
else:
    print("Digit is not 0")
```

OUTPUT:

```
Training Accuracy: 100.0
Testing Accuracy: 99.44444444444444
Digit is not 0
```

Q.3 Modify the program to detect Digit 9 vs Not Digit 9.

CODE:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
digits = load_digits()
X = digits.data
y = digits.target
# Binary classification: Digit 9 vs Not Digit 9
y = (y == 9)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
prediction = model.predict([X_test[0]])
if prediction[0]:
    print("Digit is 9")
else:
    print("Digit is not 9")
```

OUTPUT:

```
Training Accuracy: 99.68178202068417
Testing Accuracy: 98.51851851851852
Digit is not 9
```

Q.4 Modify the program to display the first 20 predictions instead of only one prediction.

CODE:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
digits = load_digits()
X = digits.data
y = digits.target
# Binary classification: Digit 5 vs Not Digit 5
y = (y == 5)
X_train, X_test, y_train, y_test = train_test_split
(X, y, test_size=0.3, random_state=1)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
# Display first 20 predictions
predictions = model.predict(X_test[:20])
actual = y_test[:20]
print("\nFirst 20 Predictions:")
for i in range(20):
    pred = "Digit 5" if predictions[i] else "Not 5"
    act = "Digit 5" if actual[i] else "Not 5"
    match = "Correct" if predictions[i] == actual[i] else "Wrong"
    print(f"{i+1}. Predicted: {pred} | Actual: {act} | {match}")
```

OUTPUT:

```
Training Accuracy: 100.0
Testing Accuracy: 98.88888888888889

First 20 Predictions:
1. Predicted: Not 5 | Actual: Not 5 | Correct
2. Predicted: Digit 5 | Actual: Digit 5 | Correct
3. Predicted: Not 5 | Actual: Not 5 | Correct
4. Predicted: Not 5 | Actual: Not 5 | Correct
5. Predicted: Not 5 | Actual: Not 5 | Correct
6. Predicted: Not 5 | Actual: Not 5 | Correct
7. Predicted: Not 5 | Actual: Not 5 | Correct
8. Predicted: Not 5 | Actual: Not 5 | Correct
9. Predicted: Digit 5 | Actual: Digit 5 | Correct
10. Predicted: Not 5 | Actual: Not 5 | Correct
11. Predicted: Not 5 | Actual: Not 5 | Correct
12. Predicted: Not 5 | Actual: Not 5 | Correct
13. Predicted: Not 5 | Actual: Not 5 | Correct
14. Predicted: Not 5 | Actual: Not 5 | Correct
15. Predicted: Not 5 | Actual: Not 5 | Correct
16. Predicted: Not 5 | Actual: Not 5 | Correct
17. Predicted: Not 5 | Actual: Not 5 | Correct
18. Predicted: Not 5 | Actual: Not 5 | Correct
19. Predicted: Not 5 | Actual: Not 5 | Correct
20. Predicted: Not 5 | Actual: Not 5 | Correct
```